
Lesson 2.1: Git



UMD DATA605 - Big Data Systems

Lesson 2.1: Git

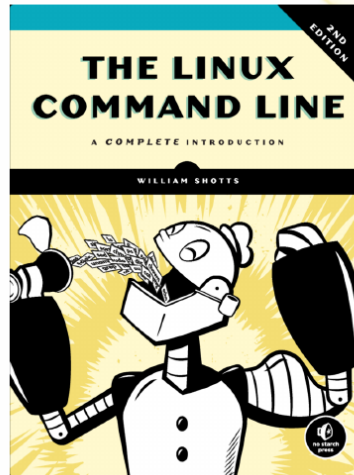
Instructor: Dr. GP Saggese, gsaggese@umd.edu



1 / 27

Bash / Linux: Resources

- **How Linux works**
 - Processes
 - File ownership and permissions
 - Virtual memory
 - How to administer a Linux box as root
- **Easy**
 - [Command-Line for Beginners](#)
 - E.g., `find`, `xargs`, `chmod`, `chown`, symbolic, and hard links
- **Mastery**
 - [The Linux Command Line](#)



2 / 27

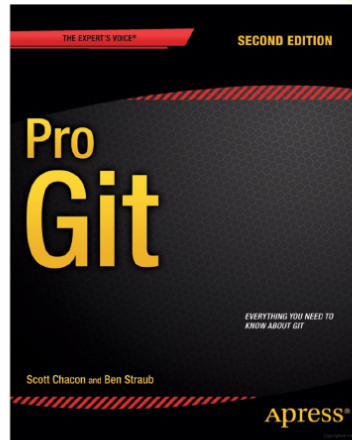
- **How Linux works**
 - *Processes*: In Linux, a process is an instance of a running program. Understanding processes is crucial because they are the basic units of execution in Linux. You can manage processes using commands like `ps`, `top`, and `kill`.
 - *File ownership and permissions*: Every file and directory in Linux has an owner and a set of permissions that determine who can read, write, or execute it. This is important for security and managing access to files.
 - *Virtual memory*: Linux uses virtual memory to extend the physical memory of the system. It allows the system to use disk space as additional RAM, which is essential for running large applications or multiple programs simultaneously.
 - *How to administer a Linux box as root*: The root user has full control over the system. Administering as root involves tasks like installing software, managing users, and configuring system settings. It's important to be cautious when operating as root to avoid accidental system damage.
- **Easy**
 - *Command-Line for Beginners*: This resource is a great starting point for those new to Linux. It covers basic commands and concepts, helping users become comfortable with the command line.
 - *E.g., `find`, `xargs`, `chmod`, `chown`, symbolic, and hard links*: These are fundamental commands and concepts in Linux. `find` helps locate files, `xargs` is used to build and execute command lines, `chmod` and `chown` are for changing file permissions and ownership, and symbolic/hard links are ways to reference files.
- **Mastery**
 - *The Linux Command Line*: This resource is for those who want to deepen their under-

standing of Linux. It covers advanced topics and provides a comprehensive guide to mastering the command line, making it ideal for users who want to become proficient in Linux.

3 / 27: Git Resources

Git Resources

- Concepts in the slides
- Tutorial: [Tutorial Git](#)
- We will use Git during the project
- Mastery: [Pro Git](#) (free)
- Web resources:
 - <https://githowto.com>
 - dangitgit.com (without swearing)
 - [Oh Sh*t, Git!?!](https://ohshitgit.com) (with swearing)
- Playgrounds
 - <https://learngitbranching.js.org>



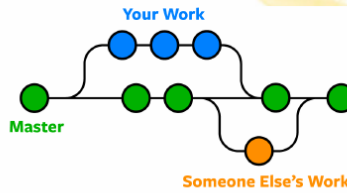
3 / 27

- **Concepts in the slides:** This bullet point suggests that the slide presentation includes key concepts related to Git, a version control system. Understanding these concepts is crucial for managing code and collaborating with others in software development projects.
- **Tutorial:** The link provided (Tutorial Git) is a resource for a Git tutorial. This tutorial is likely designed to help beginners get started with Git, covering basic commands and workflows.
- **We will use Git during the project:** This indicates that Git will be an essential tool for the upcoming project. Students should familiarize themselves with Git to effectively manage their code and collaborate with team members.
- **Mastery:** The link to Pro Git offers a free, comprehensive guide to mastering Git. This resource is ideal for those who want to deepen their understanding and become proficient in using Git.
- **Web resources:**
 - githowto.com provides step-by-step instructions for learning Git.
 - dangitgit.com offers solutions to common Git problems without using profanity.
 - Oh Sh*t, Git!?! humorously addresses Git issues with a bit of swearing, making it a fun and relatable resource.
- **Playgrounds:** learngitbranching.js.org is an interactive platform where users can practice Git commands and branching strategies in a visual and engaging way. This is particularly useful for hands-on learning and experimentation.

The images on the right side of the slide likely provide visual aids or examples related to the Git

concepts discussed, although they are not visible in this text format.

- **Branching**
 - Diverge from the main development line
- **Why branch?**
 - Work without affecting the main code
 - Avoid changes in the main branch
 - Merge code downstream for updates
 - Merge code upstream after completion
- **Git branching is lightweight**
 - Instantaneous
 - A branch is a pointer to a commit
 - Git stores data as snapshots, not file differences
- **Git workflows branch and merge often**
 - Multiple times a day
 - Surprising for users of centralized VCS
 - E.g., branch before lunch
 - Branches are cheap
 - Use them to isolate and organize work



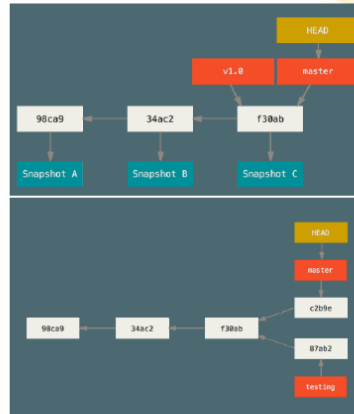
- 7 / 39

-
- Branches in Git are inexpensive to create, so it's encouraged to use them liberally to manage different tasks and features effectively.

5 / 27: Git Branching

Git Branching

- **master** (or **main**) is a normal branch
 - Pointer to the last commit
 - Moves forward with each commit
- **HEAD**
 - Pointer to the current branch
 - E.g., **master**, **testing**
 - `git checkout <BRANCH>` moves across branches
- **git branch testing**
 - Creates a new pointer **testing**
 - Points to the current commit
 - Pointer is movable
- **Divergent history**
 - Work progresses in two “split” branches



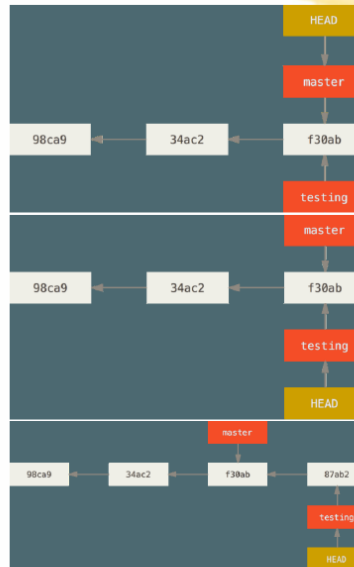
5 / 27

- **master** (or **main**) is a normal branch
 - In Git, the **master** or **main** branch is the default branch where the main line of development occurs. It's essentially a *pointer* to the latest commit in the project. As you make new commits, this pointer moves forward to include the latest changes. This is why it's often referred to as the “tip” of the branch.
- **HEAD**
 - **HEAD** is a special pointer in Git that indicates the current branch you are working on. For example, if you are working on the **master** branch, **HEAD** points to **master**. When you switch branches using `git checkout <BRANCH>`, **HEAD** moves to point to the new branch, allowing you to work on different parts of your project without affecting the main line of development.
- **git branch testing**
 - This command creates a new branch named **testing**. It acts as a new pointer that starts at the current commit where **HEAD** is located. This new branch is independent and can be moved forward as you make new commits, allowing you to experiment or develop features without affecting the main branch.
- **Divergent history**
 - When you create a new branch and start making changes, the history of commits can diverge. This means that the **master** branch and the **testing** branch can have different sets of commits, representing different lines of development. This is useful for working on features or fixes in isolation before merging them back into the main branch.

6 / 27: Git Checkout

Git Checkout

- `git checkout` switches branches
 - Moves `HEAD` pointer to the new branch
 - Changes files in the working directory to match the branch pointer
- E.g., two branches, `master` and `testing`
 - You are on `master`
 - `git checkout testing`
 - Pointer moves, working directory changes
 - Keep working and commit on `testing`
 - Pointer to `testing` moves forward



6 / 27

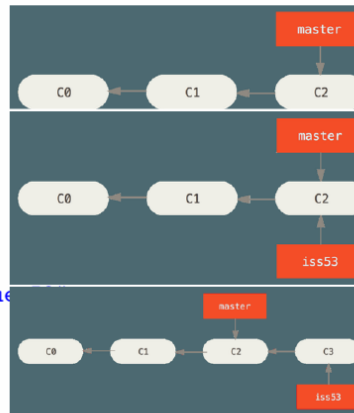
- `git checkout` switches branches
 - The `git checkout` command is used to switch between different branches in a Git repository. This is a common operation when you want to work on different features or versions of your project.
 - When you use `git checkout`, it moves the `HEAD` pointer to the branch you specify. The `HEAD` is a reference to the current branch you are working on.
 - It also updates the files in your working directory to match the state of the branch you switched to. This means that any changes or files that are specific to the new branch will be reflected in your working directory.
- **Example with two branches: `master` and `testing`**
 - Imagine you have two branches in your project: `master` and `testing`.
 - If you are currently on the `master` branch and you want to switch to the `testing` branch, you would use the command `git checkout testing`.
 - This action moves the `HEAD` pointer from `master` to `testing`, and your working directory will now reflect the files and changes that are part of the `testing` branch.
 - You can continue to work on the `testing` branch, making changes and committing them. Each commit will move the pointer of the `testing` branch forward, capturing the new state of your work.
- **Visual aids**
 - The images on the right side of the slide likely illustrate the process of switching branches and how the `HEAD` pointer and working directory change. These visuals can help you understand the concept of branch switching in a more intuitive way.

7 / 27: Git Branching and Merging

Git Branching and Merging

- Tutorials
 - [Work on main](#)
 - [Hot fix](#)
- Start from a project with some commits
- Branch to work on a new feature “Issue 53”

```
> git checkout -b iss53
work ... work ... work
> git commit -m "Add feature for Issue 53"
```

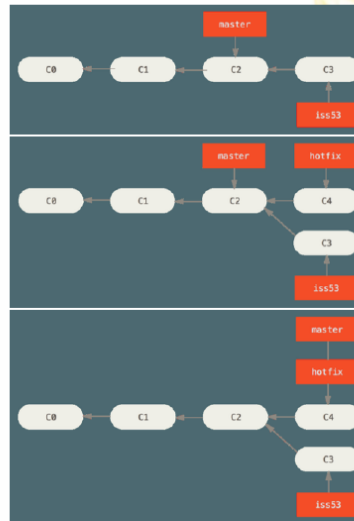


- **Git Branching and Merging:** This slide is about using Git, a version control system, to manage changes in a project. Specifically, it focuses on branching and merging, which are key features of Git that help in organizing and integrating different lines of work.
- **Tutorials:** The slide provides links to tutorials that guide you through working on the main branch and handling hot fixes. These tutorials are useful for understanding how to manage the main line of development and quickly address urgent issues without disrupting ongoing work.
- **Start from a project with some commits:** Before branching, you should have a project with existing commits. This means you have a history of changes that have been saved, which provides a foundation for creating new branches.
- **Branch to work on a new feature “Issue 53”:** The slide demonstrates how to create a new branch for a specific task or feature, in this case, “Issue 53”. By using the command `git checkout -b iss53`, you create and switch to a new branch named **iss53**. This allows you to work on the feature independently without affecting the main project. After making changes, you commit them with a message like “Add feature for Issue 53” to document what was done.
- **Images:** The images likely illustrate the branching and merging process visually, showing how branches diverge from the main line and are eventually merged back. This visual representation helps in understanding the flow of changes and how different branches relate to each other.

8 / 27: Git Branching and Merging

Git Branching and Merging

- **Need a hotfix** to master
 - > `git checkout master`
 - > `git checkout -b hotfix`
 - `fix ... fix ... fix`
 - > `git commit -am "Hot fix"`
 - > `git checkout master`
 - > `git merge hotfix`
- **Fast forward**
 - Now there is no divergent history between `master` and `iss53`



8 / 27

- **Need a hotfix** to master
 - This section explains how to apply a quick fix, known as a *hotfix*, directly to the `master` branch in Git.
 - The process begins by switching to the `master` branch using `git checkout master`.
 - A new branch named `hotfix` is created from `master` with `git checkout -b hotfix`. This allows you to work on the fix without affecting the `master` branch until it's ready.
 - After making the necessary changes, you commit them with `git commit -am "Hot fix"`. The `-am` flag is used to add and commit changes with a message in one step.
 - Finally, you switch back to the `master` branch and merge the `hotfix` branch into it using `git merge hotfix`. This incorporates the hotfix changes into `master`.
- **Fast forward**
 - This term refers to a specific type of merge in Git where the `master` branch is directly updated to match another branch, like `iss53`, without creating a new commit.
 - A fast forward merge happens when there is no divergent history between the branches, meaning the `master` branch can simply be moved forward to the latest commit of the other branch.
 - This is a clean and simple way to update branches when no conflicts or additional history need to be resolved.

9 / 27: Git Branching and Merging

Git Branching and Merging

- Keep working on `iss53`

```
> git checkout iss53
```

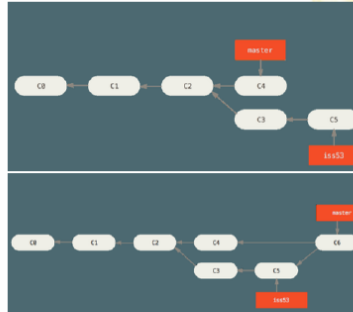

`work ... work ... work`
 - The branch keeps diverging
- At some point you are done with `iss53`
 - You want to merge your work back to `master`
 - Go to the target branch

```
> git checkout master
```



```
> git merge iss53
```
- Git can't fast forward
- Git creates a new snapshot with the 3-way "merge commit" (i.e., a commit with more than one parent)
- Delete the branch

```
> git branch -d iss53
```



9 / 27

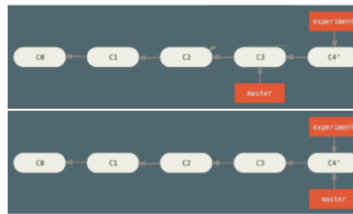
- **Keep working on `iss53`**
 - This step involves switching to the branch named `iss53` using the command `git checkout iss53`. Once on this branch, you continue making changes and committing them. This is a common practice when working on a specific feature or issue, allowing you to isolate your work from the main codebase.
 - **The branch keeps diverging:** As you work on `iss53`, the branch may diverge from the `master` branch if other changes are made to `master` in the meantime. This means the two branches have different histories.
- **At some point you are done with `iss53`**
 - Once your work on `iss53` is complete, you need to integrate these changes back into the main branch, typically `master`. This is done by first switching to the `master` branch using `git checkout master`.
 - **Go to the target branch:** After switching to `master`, you merge the changes from `iss53` into `master` using `git merge iss53`.
- **Git can't fast forward**
 - If Git cannot perform a fast-forward merge, it means that the `master` branch has progressed since `iss53` was created. In such cases, Git creates a new commit called a "merge commit" to combine the histories of both branches.
- **Git creates a new snapshot with the 3-way "merge commit"**
 - A merge commit is a special type of commit that has more than one parent. It represents the point where two branches have been combined, preserving the history of both branches.
- **Delete the branch**
 - After successfully merging `iss53` into `master`, the branch `iss53` is no longer needed. You

can delete it using `git branch -d iss53` to keep your branch list clean and organized. This step is important for maintaining a tidy repository and avoiding clutter from unused branches.

10 / 27: Fast Forward Merge

Fast Forward Merge

- Merge a commit Y with a commit X that can be reached by following the history of commit Y
- There is no divergent history to merge
 - Git simply moves the branch pointer forward from X to Y
- **Mental model:** a branch is just a pointer that indicates where the tip of the branch is
- E.g., C4' is reachable from C3
 - > `git checkout master`
 - > `git merge experiment`
- Git moves the pointer of master to C4'



10 / 27

- **Fast Forward Merge:** This is a type of merge in Git where you combine two commits, Y and X, but X is already part of the history of Y. This means that Y is a direct descendant of X.
- **No Divergent History:** In this scenario, there are no separate paths or branches that need to be reconciled. Since Y is directly reachable from X, Git doesn't need to create a new commit to merge them. Instead, it simply updates the branch pointer to point to Y.
- **Mental Model:** Think of a branch as a label or pointer that marks the latest commit in that branch. When you perform a fast forward merge, you're essentially just moving this label forward to the latest commit.
- **Example:** If you have a branch `master` and another branch `experiment`, and `experiment` is ahead of `master` without any conflicting changes, you can merge them with:

```
> git checkout master
> git merge experiment
```

This command moves the `master` branch pointer to the latest commit on `experiment`, effectively making `master` up-to-date with `experiment`.

- **Visual Representation:** The images likely show how the branch pointer moves from one commit to another, illustrating the simplicity of a fast forward merge. This visual aid helps in understanding that no new commits are created during this process.

11 / 27: Merging Conflicts

Merging Conflicts

- Tutorial:
 - [Merging conflicts](#)
- Sometimes **Git can't merge**, e.g.,
 - The same file has been modified by both branches
 - One file was modified by one branch and deleted by another
- **Git:**
 - Does not create a merge commit
 - Pauses to let you resolve the conflict
 - Adds conflict resolution markers
- **User merges manually**
 - Edit the files using `git mergetool`
 - Use `git add` to mark as resolved
 - Use `git commit` to finalize the merge
 - Use PyCharm or VS Code for assistance

```
<<<<<< HEAD:index.html
<div id="footer">contact : email.support@github.com</div>
=====
<div id="footer">
  please contact us at support@github.com
</div>
>>>>>> iss53:index.html

$ git merge iss53
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")

Unmerged paths:
  (use "git add <file>..." to mark resolution)

    both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   index.html
```



11 / 27

- **Merging Conflicts:**
 - This slide is about handling situations where *Git*, a version control system, encounters conflicts during a merge. Merging is when you combine changes from different branches of a project. Sometimes, Git can't automatically merge changes because of conflicts.
- **Tutorial:**
 - A tutorial is provided to help understand merging conflicts. It can be found here. This resource is useful for learning how to handle these situations effectively.
- **When Git Can't Merge:**
 - Conflicts occur when the same file is modified in different ways on two branches. For example, if both branches change the same line in a file, Git won't know which change to keep.
 - Another common conflict is when one branch modifies a file while another branch deletes it. Git can't decide whether to keep the changes or delete the file.
- **Git's Behavior During Conflicts:**
 - Git will not automatically create a merge commit when it encounters a conflict. Instead, it pauses the process to allow the user to resolve the conflict manually.
 - It adds conflict resolution markers in the files to show where the conflicts are. These markers help users identify the conflicting changes.
- **Manual Conflict Resolution:**
 - Users need to manually edit the files to resolve conflicts. This can be done using tools like `git mergetool`, which helps in comparing and editing the conflicting files.
 - After resolving the conflicts, users must use `git add` to mark the conflicts as resolved.
 - Finally, a `git commit` is needed to complete the merge process.
 - Integrated development environments (IDEs) like PyCharm or VS Code can assist in

resolving conflicts by providing visual tools to compare and edit files.

12 / 27: Git Rebasing

Git Rebasing

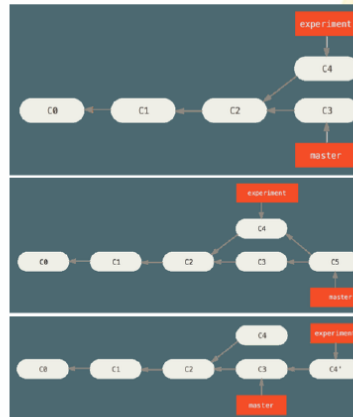
- In Git, there are **two ways of merging divergent history**
- E.g., consider master and experiment have a common ancestor C2

- **Merge**

- Go to the target branch
 - > `git checkout master`
 - > `git merge experiment`
- Create a new snapshot C5 and commit

- **Rebase**

- Go to the branch to rebase
 - > `git checkout experiment`
 - > `git rebase master`
- **Rebase algorithm:**
 - Get all the changes committed in the branch (C4) where we are on (experiment) since the common ancestor (C2)
 - Sync to the branch that we are rebasing onto (master at C3)
 - Apply the changes C4
 - Only the current branch is affected
 - Finally, fast forward experiment

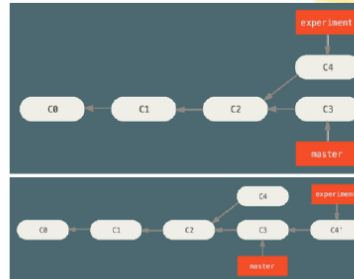


12 / 27

- **Two Ways of Merging Divergent History in Git**
 - In Git, when you have two branches that have diverged from a common point, you can bring them together using either a *merge* or a *rebase*. This slide explains these two methods using an example with branches **master** and **experiment**, which share a common ancestor C2.
- **Merge**
 - To merge, you first switch to the target branch, in this case, **master**.
 - You then execute the `git merge experiment` command. This creates a new commit, C5, which combines the changes from both branches. This method keeps the history of both branches intact, showing a clear path of how the branches came together.
- **Rebase**
 - Rebasing involves switching to the branch you want to rebase, here **experiment**.
 - The `git rebase master` command is used. This process involves taking all the changes made in **experiment** since the common ancestor C2 and applying them on top of **master** at C3.
 - This method rewrites the commit history, making it appear as though the changes from **experiment** were made after **master**, resulting in a linear history. Only the current branch (**experiment**) is affected, and it is fast-forwarded to include the changes.
- **Visual Aids**
 - The images on the right side of the slide likely illustrate the differences in commit history between merging and rebasing, helping to visualize how each method affects the branch structure.

Uses of Rebase

- **Rebasing makes for a cleaner history**
 - The history looks like all the work happened in series
 - Although in reality, it happened in parallel to the development in the master branch
- **Rebasing to contribute to a project**
 - Developer
 - You are contributing to a project that you don't maintain
 - You work on your branch
 - When you are ready to integrate your work, rebase your work onto origin/master
 - The maintainer
 - Does not have to do any integration work
 - Does just a fast forward or a clean apply (no conflicts)

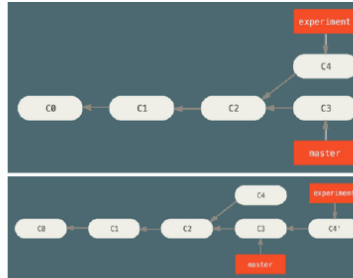


- **Rebasing makes for a cleaner history**
 - When you rebase, it rearranges the commit history to appear as if all changes were made one after the other, rather than simultaneously. This creates a linear and easy-to-follow history.
 - In reality, multiple developers might be working on different features or fixes at the same time, but rebasing makes it look like these changes were made sequentially. This can make it easier to understand the progression of the project over time.
- **Rebasing to contribute to a project**
 - *Developer:* If you're contributing to a project that you don't own, you typically work on a separate branch. Once your work is complete, you rebase your branch onto the latest version of the main branch (often called **origin/master**). This ensures your changes are up-to-date with the main project.
 - *The maintainer:* By rebasing, you make it easier for the project maintainer to incorporate your changes. They can simply fast forward the main branch to include your work, or apply your changes without dealing with conflicts, making the integration process smoother and more efficient.

14 / 27: Golden Rule of Rebasing

Golden Rule of Rebasing

- **Remember:** rebasing means abandoning existing commits and creating new ones that are similar but different
- **Problem**
 - You push commits to a remote
 - Others pull commits and base work on them
 - You rewrite commits with `git rebase`
 - You push again with `git push --force`
 - Collaborators must re-merge work
- **Solution**
 - Strict: *"Do not ever rebase commits outside your repository"*
 - Loose: *"Rebase your branch if only you use it, even if pushed to a server"*



14 / 27

- **Golden Rule of Rebasing**
 - **Remember:** When you rebase, you are essentially taking existing commits and creating new ones that are similar but not identical. This means the history of your project changes, which can lead to complications if not handled carefully.
- **Problem**
 - **You push commits to a remote:** This means you've shared your work with others, and they might start using it as a base for their own work.
 - **Others pull commits and base work on them:** Your collaborators download your changes and start building their own work on top of it.
 - **You rewrite commits with `git rebase`:** Rebasing changes the commit history, which can make the history look cleaner but also changes the commit IDs.
 - **You push again with `git push --force`:** This action forces the new commit history onto the remote, overwriting the old history.
 - **Collaborators must re-merge work:** Because the commit history has changed, your collaborators will have to adjust their work to fit the new history, which can be time-consuming and error-prone.
- **Solution**
 - **Strict:** *"Do not ever rebase commits outside your repository"*: This means you should avoid rebasing commits that have already been shared with others to prevent disrupting their work.
 - **Loose:** *"Rebase your branch if only you use it, even if pushed to a server"*: If you are the only one working on a branch, it's generally safe to rebase, even if the branch is on a remote server, because no one else is affected by the changes.

The images likely illustrate the difference between a clean commit history with rebasing and a

more complex one without it, emphasizing the importance of understanding when and how to use rebasing effectively.

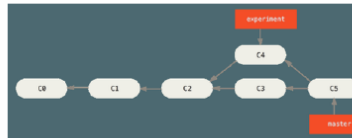
15 / 27: Rebase vs Merge: Philosophical Considerations

Rebase vs Merge: Philosophical Considerations

- Deciding **Rebase-vs-merge** depends on the answer to the question:
 - What does the commit history of a repo mean?

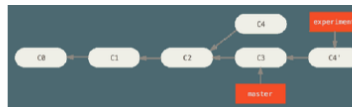
1. **History is the record of what actually happened**

- "History should not be tampered with, even if messy!"
- Use `git merge`



2. **History represents how a project should have been made**

- "You should tell the history in the way that is best for future readers"
- Use `git rebase` and `filter-branch`



15 / 27

- Rebase vs Merge: Philosophical Considerations**
 - The choice between **Rebase** and **Merge** is not just a technical decision but also a philosophical one. It revolves around how you perceive the commit history of a repository. The key question to ask is: *What does the commit history of a repo mean?*
- History is the record of what actually happened**
 - This perspective values authenticity and transparency. The idea is that the commit history should reflect the true sequence of events, even if it appears messy or complicated. This approach is akin to keeping a raw, unedited diary.
 - If you align with this view, you would typically use `git merge`. Merging keeps all the original commits intact, showing exactly how the project evolved over time, including all branches and merges.
- History represents how a project should have been made**
 - This viewpoint prioritizes clarity and readability for future developers. It suggests that the commit history should be curated to present the most logical and understandable sequence of events, even if it means altering the original order.
 - If you prefer this approach, you would use `git rebase` and possibly `filter-branch`. Rebasing allows you to rewrite commit history, creating a linear and cleaner narrative that can be easier for others to follow and understand.

In summary, the decision between rebase and merge is about choosing between preserving the authentic history of a project or crafting a more streamlined and comprehensible version for future reference.

16 / 27: Rebase vs Merge: Philosophical Considerations

Rebase vs Merge: Philosophical Considerations

- **Many man-centuries have been wasted** discussing rebase-vs-merge at the watercooler
 - Total waste of time! Tell people to get back to work!
- When you contribute to a project often people decide for you based on their preference
- **Best of the merge-vs-rebase approaches**
 - Rebase changes you've made in your local repo
 - Even if you have pushed but you know the branch is yours
 - Use `git pull --rebase` to clean up the history of your work
 - If the branch is shared with others then you need to definitively `git merge`
 - Only `git merge` to master to preserve the history of how something was built
- **Personally**
 - I like to squash-and-merge branches to master
 - Rarely are my commits “complete”; they are just checkpoints



16 / 27

- **Many man-centuries have been wasted** discussing rebase-vs-merge at the watercooler
 - This point humorously highlights how much time developers spend debating whether to use rebase or merge in version control systems like Git. The phrase “man-centuries” exaggerates the time spent, suggesting it’s a common and often unproductive discussion. The advice to “get back to work” implies that these debates can be a distraction from actual coding tasks.
- When you contribute to a project often people decide for you based on their preference
 - In many projects, the choice between rebase and merge isn’t up to the individual developer. Instead, it’s determined by the project’s existing practices or the preferences of the project maintainers. This means that developers often have to adapt to the workflow that is already in place.
- **Best of the merge-vs-rebase approaches**
 - *Rebase changes you’ve made in your local repo*
 - * Rebasing is useful for keeping your local changes up-to-date with the main branch. Even if you’ve already pushed your changes, you can rebase if the branch is solely yours. This helps maintain a clean and linear project history.
 - * Using `git pull --rebase` is a command that integrates changes from the main branch into your local branch, making the history cleaner by avoiding unnecessary merge commits.
 - * If your branch is shared with others, merging is safer to avoid disrupting the shared history.
 - *Only `git merge` to master to preserve the history of how something was built*
 - * Merging into the master branch is recommended to keep a complete and accurate history of how the project evolved. This approach helps in understanding the sequence

of changes and the context in which they were made.

- **Personally**

- *I like to squash-and-merge branches to **master***

- * Squash-and-merge is a technique where multiple commits are combined into a single commit before merging into the master branch. This can simplify the history by reducing clutter from numerous small commits.

- *Rarely are my commits “complete”; they are just checkpoints*

- * This personal note suggests that the speaker’s commits are often incremental and not fully polished. They use commits as checkpoints to save progress, which is why they prefer to squash them before merging to keep the main branch clean and organized.

17 / 27: Remote Branches

Remote Branches

- **Remote branches** are pointers to branches in remote repositories

```
> git remote -v
origin  git@github.com:gpsaggese/umd_classes.git (fetch)
origin  git@github.com:gpsaggese/umd_classes.git (push)
```

- **Tracking branches**

- Local references representing the state of the remote repository
- E.g., `master` tracks `origin/master`
- You can't change the remote branch (e.g., `origin/master`)
- You can change the tracking branch (e.g., `master`)
- Git updates tracking branches when you do `git fetch origin` (or `git pull`)

- To share code in a local branch you need to push it to a remote

```
> git push origin serverfix
```

- To work on it

```
> git checkout -b serverfix origin/serverfix
```



SCIENCE
ACADEMY

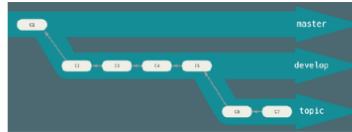
17 / 27

- **Remote branches** are essentially bookmarks or pointers that represent branches in a remote repository. They help you keep track of what the state of the branches is on the remote server. When you run the command `git remote -v`, it shows you the URLs of the remote repositories you have configured, both for fetching and pushing changes. This is crucial for collaboration, as it allows multiple developers to work on the same project from different locations.
- **Tracking branches** are local branches that have a direct relationship with a remote branch. For example, your local `master` branch might track the `origin/master` branch on the remote. This means that your local branch is set up to pull changes from and push changes to the remote branch. You cannot directly modify a remote branch like `origin/master`; instead, you make changes to your local tracking branch and then push those changes to the remote. When you run `git fetch origin` or `git pull`, Git updates your tracking branches to reflect the latest state of the remote branches.
- To share your work with others, you need to push your local branch to the remote repository. For instance, if you have a local branch called `serverfix`, you can share it by pushing it to the remote with the command `git push origin serverfix`. This makes your changes available to others who have access to the remote repository.
- If you want to start working on a branch that exists on the remote but not locally, you can create a new local branch that tracks the remote branch. The command `git checkout -b serverfix origin/serverfix` does exactly this. It creates a new local branch called `serverfix` that tracks the `serverfix` branch on the remote, allowing you to work on it and later push your changes back to the remote.

18 / 27: Git Workflows

Git Workflows

- **Git workflows** = ways of working and collaborating using Git
- **Long-running branches** = branches at different levels of stability that are always open
 - **master** is always ready to be released
 - **develop** branch to develop in
 - **topic/feature branches**
 - When branches are “stable enough,” they are merged up

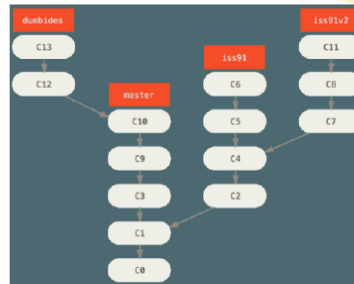


18 / 27

- **Git workflows:** This refers to the different methods or strategies teams use to collaborate and manage code changes using Git. Git is a version control system that helps track changes in code, making it easier for multiple people to work on the same project without conflicts.
- **Long-running branches:** These are branches in a Git repository that remain open for extended periods. They serve different purposes and have varying levels of stability:
 - **master branch:** This is the main branch that is always ready for release. It contains the most stable version of the code.
 - **develop branch:** This branch is used for ongoing development. It is where new features and changes are integrated before they are stable enough to be merged into the **master**.
 - **Topic/feature branches:** These are temporary branches created to work on specific features or fixes. Once the work is complete and tested, these branches are merged into the **develop** branch.
 - **Merging:** When a branch is considered “stable enough,” it is merged into a higher-level branch, like **develop** or **master**, to ensure the main codebase remains stable and up-to-date.

Git Workflows

- **Topic branches** = short-lived branches for a single feature
 - E.g., hotfix, wip-XYZ
 - Easy to review
 - Siloed from the rest
 - This is typical of Git since other VCS support for branches is not good enough
 - E.g.,
 - You start `iss91`, then you cancel some stuff, and go to `iss91v2`
 - Somebody starts a `dumbidea` branch and merges it to `master` (!)
 - You `squash-and-merge` your `iss91v2`

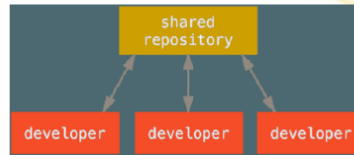


- **Topic branches** are a key feature in Git that allow developers to create *short-lived branches* for working on a single feature or fix. These branches are temporary and are used to isolate work until it is ready to be integrated into the main codebase.
 - For example, branches like `hotfix` or `wip-XYZ` are created to address specific issues or work-in-progress tasks.
 - These branches are *easy to review* because they contain changes related to a single feature or fix, making it simpler for team members to understand and provide feedback.
 - They are *siloed from the rest* of the codebase, meaning changes in these branches do not affect the main code until they are merged. This isolation helps prevent unfinished or experimental code from disrupting the stable version.
 - Git excels at handling branches, unlike some other version control systems (VCS) where branch support might not be as robust.
 - For instance, you might start working on a branch named `iss91`, realize some changes need to be discarded, and then continue on a new branch `iss91v2`.
 - Sometimes, a branch like `dumbidea` might be created for experimental purposes and mistakenly merged into the main branch (`master`), which can lead to issues if not managed carefully.
 - When you are satisfied with the changes in your branch, you can use a technique called *squash-and-merge* to combine all the changes from `iss91v2` into a single commit before merging it into the main branch. This helps keep the commit history clean and organized.

Centralized Workflow

- **Centralized workflow in centralized VCS**

- Developers:
 - Check out the code from the central repo on their computer
 - Modify the code locally
 - Push it back to the central hub (assuming no conflicts with the latest copy, otherwise they need to merge)



- **Centralized workflow in Git**

- Developers:
 - Have push (i.e., write) access to the central repo
 - Need to fetch and then merge
 - Cannot push code that will overwrite each other's code (only fast-forward changes)



20 / 27

- **Centralized workflow in centralized VCS**

- *Developers:*

- * **Check out the code from the central repo on their computer:** This means that developers download the latest version of the code from a central repository to their local machine. This is the starting point for any changes they want to make.
- * **Modify the code locally:** Developers work on their own computers to make changes to the code. This allows them to test and develop features independently.
- * **Push it back to the central hub:** Once changes are complete, developers upload their modified code back to the central repository. If someone else has changed the code in the meantime, they may need to resolve conflicts before their changes can be accepted.

- **Centralized workflow in Git**

- *Developers:*

- * **Have push (i.e., write) access to the central repo:** This means developers can directly upload their changes to the central repository, but they need permission to do so.
- * **Need to fetch and then merge:** Before pushing changes, developers must first download the latest version of the code (fetch) and integrate any new changes into their work (merge). This ensures that their changes are compatible with the latest version.
- * **Cannot push code that will overwrite each other's code:** Git prevents developers from overwriting each other's work by only allowing changes that can be fast-forwarded. This means that changes must be added in a way that builds on the existing code, rather than replacing it.

21 / 27: Forking Workflows

Forking Workflows

- Typically, developers don't have permissions to update branches directly on a project
 - Read-write permissions for core contributors
 - Read-only for everybody else
- **Solution**
 - "Forking" a repo
 - External contributors:
 - Clone the repo and create a branch with their work
 - Create a writable fork of the project
 - Push branches to the fork
 - Prepare a PR (Pull Request) with their work
 - Project maintainer:
 - Reviews PRs
 - Accepts PRs
 - Integrates PRs
 - In practice, it's the project maintainer who pulls the code when it's ready, instead of external contributors pushing the code
- **Aka "GitHub workflow"**
 - The "innovation" was forking (Fork me on GitHub!)
 - GitHub was acquired by Microsoft for 7.5 billion USD



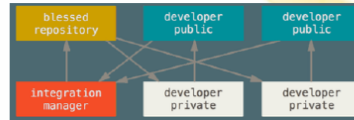
21 / 27

- **Forking Workflows**
 - In many open-source projects, not all developers have the ability to make changes directly to the project's code. This is because only a select group of core contributors have read-write permissions, meaning they can make changes directly. Everyone else has read-only access, which means they can view the code but cannot make changes directly.
- **Solution**
 - The solution to this limitation is a process called "forking." Forking allows developers to create their own copy of the project where they can make changes freely.
 - **External contributors:**
 - * They start by cloning the repository, which means they make a local copy of the project on their computer.
 - * They then create a writable fork, which is essentially their own version of the project on the platform (like GitHub).
 - * They can create branches in their fork to work on new features or bug fixes.
 - * Once they are satisfied with their changes, they prepare a Pull Request (PR) to propose their changes to the original project.
 - **Project maintainer:**
 - * The maintainer reviews these PRs to ensure the changes are beneficial and do not introduce issues.
 - * If the changes are approved, the maintainer accepts and integrates them into the main project.
 - * This process ensures that the project maintainer has control over what changes are made to the project.
- **Aka "GitHub workflow"**

-
- This workflow is often referred to as the “GitHub workflow” because GitHub popularized the concept of forking with their platform. The phrase “Fork me on GitHub!” became a symbol of this innovation.
 - GitHub’s success with this model was significant enough that Microsoft acquired the company for 7.5 billion USD, highlighting the value and impact of this approach to collaborative software development.

Integration-Manager Workflow

- This is the classical model for open-source development
 - E.g., Linux, GitHub (forking) workflow



1. **One repo is the official project**
 - Only the project maintainer pushes to the public repo
 - E.g., `causify-ai/csfy`
2. **Each contributor**
 - Has read access to everyone else's public repo
 - Forks the project into a private copy
 - Write access to their own public repo
 - E.g., `gpsaggese/csfi`
 - Makes changes
 - Pushes changes to their own public copy
 - Sends a pull request to the maintainer asking to merge changes
3. **The maintainer**
 - Adds the contributor's repo as a remote
 - Merges the changes into a local branch
 - Tests changes locally
 - Pushes the branch to the official repo



22 / 27

- **Integration-Manager Workflow**

This workflow is a traditional model used in open-source development. It's commonly seen in projects like Linux and on platforms like GitHub, where the concept of forking is prevalent. This model helps manage contributions from multiple developers efficiently.

- **One repo is the official project**

In this workflow, there is a single official repository that represents the project. The project maintainer is the only person who has the authority to push changes directly to this public repository. For example, in a project named `causify-ai/csfi`, the maintainer would be responsible for managing the official repo.

- **Each contributor**

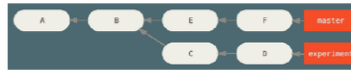
Contributors have read access to the public repositories of others, allowing them to see what changes are being made. They create a personal copy of the project, known as a fork, where they have write access. For instance, a contributor might fork the project to `gpsaggese/csfi`. They make their changes in this fork and then push these changes to their own public repository. Once satisfied, they send a pull request to the maintainer, requesting that their changes be merged into the official project.

- **The maintainer**

The maintainer plays a crucial role in this workflow. They add the contributor's repository as a remote to their local setup, allowing them to pull in the changes. The maintainer then merges these changes into a local branch and tests them to ensure everything works correctly. Once the changes are verified, the maintainer pushes the updated branch to the official repository, integrating the contributor's work into the project. This process ensures that only stable and tested changes make it into the official project.

Git log

- `git log` reports info about commits
- **refs** are references to:
 - HEAD (commit you are working on, next commit)
 - origin/master (remote branch)
 - experiment (local branch)
 - d921970 (commit)
- `^` after a reference resolves to the parent of that commit
 - HEAD[^] = commit before HEAD, i.e., last commit
 - `^2` means the second parent of a merge commit
 - A merge commit has multiple parents



23 / 27

- **Git log:** The `git log` command is a powerful tool in Git that provides detailed information about the history of commits in a repository. It allows you to see what changes have been made, who made them, and when they were made. This is crucial for understanding the evolution of a project and for tracking down when specific changes were introduced.
- **Refs (References):** In Git, references or “refs” are pointers to commits. They help you navigate through the history of your project:
 - **HEAD:** This is the current commit you are working on. It points to the latest commit in your current branch and is where your next commit will be based.
 - **origin/master:** This refers to the remote branch, typically the main branch on the remote repository. It helps you keep track of what the main branch looks like on the server.
 - **experiment:** This is an example of a local branch. Local branches are branches you create on your machine to work on features or fixes.
 - **d921970:** This is an example of a specific commit identifier (SHA-1 hash). Each commit in Git has a unique identifier that allows you to reference it directly.
- **Caret (^) Notation:** The caret symbol is used to navigate through commit history:
 - **HEAD[^]:** This refers to the commit immediately before the current HEAD. It’s useful for looking at the previous state of your project.
 - **`^2`:** This is used in the context of merge commits, which have more than one parent. `^2` refers to the second parent of a merge commit, allowing you to see the other branch that was merged.
 - **Merge Commits:** These are special commits that combine changes from different

branches. They have multiple parents, which is why the caret notation is useful for exploring them.

The image on the slide likely illustrates these concepts visually, showing how refs and caret notation help navigate the commit history.

24 / 27: Dot notation

Dot notation

- **Double-dot notation**

- `1..2` = commits that are reachable from 2 but not from 1
- Like a “difference”
- `git log master..experiment` → D,C
- `git log experiment..master` → F,E



- **Triple-dot notation**

- `1...2` = commits that are reachable from either branch but not from both
- Like “union excluding intersection”
- `git log master...experiment` → F,E,D,C



24 / 27

- **Double-dot notation**

- The double-dot notation in Git is used to compare two branches or commits. When you see something like `1..2`, it means you are looking at commits that are reachable from the second commit (or branch) but not from the first. Think of it as finding the “difference” between two points in your project history.
- For example, if you run `git log master..experiment`, Git will show you commits D and C. These are the changes that exist in the **experiment** branch but not in the **master** branch.
- Conversely, `git log experiment..master` will show you commits F and E, which are present in **master** but not in **experiment**.

- **Triple-dot notation**

- The triple-dot notation is a bit different. When you see `1...2`, it refers to commits that are reachable from either of the two branches or commits, but not from both. It’s like taking a “union” of the two sets of commits and then excluding the ones that are common to both.
- For instance, `git log master...experiment` will list commits F, E, D, and C. These are all the commits that are unique to either **master** or **experiment**, excluding any that might be common to both branches.

Advanced Git



- **Stashing**
 - Copy the state of your working directory (e.g., modified and staged files)
 - Save it in a stack
 - Apply it later
- **Cherry-picking**
 - Apply a single commit from one branch onto another
- **rerere**
 - = “Reuse Recorded Resolution”
 - Git caches how to solve certain conflicts
- **Submodules / subtrees**
 - Projects including other Git projects



25 / 27

- Advanced Git
- **Stashing**
 - *Stashing* is a useful feature in Git that allows you to temporarily save changes in your working directory. This includes both modified and staged files. Think of it as putting your work on hold without committing it. This is particularly helpful when you need to switch branches or work on something else without losing your current progress. The changes are saved in a stack, which means you can have multiple stashes and apply them later when you’re ready to continue working.
- **Cherry-picking**
 - *Cherry-picking* is a powerful tool that lets you apply a specific commit from one branch to another. This is useful when you want to incorporate a particular change without merging entire branches. It allows for selective integration of changes, which can be handy in managing features or bug fixes across different branches.
- **rerere**
 - The term *rerere* stands for “Reuse Recorded Resolution.” This feature in Git helps by remembering how you resolved conflicts in the past. When similar conflicts arise again, Git can automatically apply the same resolution, saving you time and effort. It’s particularly beneficial in projects with frequent merges and recurring conflicts.
- **Submodules / subtrees**
 - These are methods to include one Git project within another. *Submodules* allow you to

keep a separate repository within your main project, maintaining its own history and independence. *Subtrees*, on the other hand, integrate the external project more tightly, allowing for easier management of changes. Both are useful for managing dependencies or incorporating external libraries into your project.

Advanced Git

- **bisect**
 - `git bisect` helps identify the commit that introduced a bug
 - Bug appears at the top of the tree
 - Unknown revision where it started
 - Script returns 0 if good, non-zero if bad
 - `git bisect` finds the revision where the script changes from good to bad
- **filter-branch**
 - Rewrite repository history in a scriptable way
 - E.g., change email, remove sensitive file
 - Check out each version, run a command, commit the result
- **Hooks**
 - Run scripts before committing, merging, etc



26 / 27

- **bisect**
 - *Purpose:* `git bisect` is a powerful tool used to pinpoint the exact commit that introduced a bug in your codebase.
 - * Imagine your code is like a tree, and the bug is at the top. You know it's there, but you're not sure when it first appeared.
 - * You start with a known good revision and a bad one (where the bug is present), but the exact point of introduction is unknown.
 - * You can automate the process by using a script that tests each commit. The script should return 0 if the commit is good (bug-free) and a non-zero value if it's bad (bug present).
 - * `git bisect` will systematically narrow down the range of commits to find the first bad commit, saving you time and effort.
- **filter-branch**
 - *Purpose:* This command allows you to rewrite the history of your repository in a flexible, scriptable manner.
 - * For example, you might need to change an author's email address across all commits or remove a file containing sensitive information.
 - * The process involves checking out each commit, applying your changes (like running a command), and then committing the result back into the history.
 - * It's a powerful tool but should be used with caution, as rewriting history can affect collaborators.
- **Hooks**
 - *Purpose:* Hooks are scripts that Git can run automatically at certain points in your workflow, such as before committing or merging.

-
- * They allow you to automate tasks and enforce policies, like running tests before a commit or preventing certain types of merges.
 - * Hooks can help maintain code quality and consistency across your team by ensuring that certain conditions are met before changes are accepted.

GitHub

- GitHub acquired by MSFT for \$7.5 billion
- **GitHub: largest host for Git repositories**
 - Git hosting (100M+ open source projects)
 - Pull Requests (PRs), forks
 - Issue tracking
 - Code review
 - Collaboration
 - Wiki
 - Actions (CI/CD)
- **"Forking a project"**
 - Open-source communities
 - Negative connotation
 - Modify and create a competing project
 - GitHub parlance
 - Copy a project to contribute without push/write access



- **GitHub acquired by MSFT for \$7.5 billion**
 - In 2018, Microsoft acquired GitHub, a significant move given GitHub's role as a major platform for software development. This acquisition was valued at \$7.5 billion, highlighting GitHub's importance in the tech industry.
- **GitHub: largest host for Git repositories**
 - GitHub is the largest platform for hosting Git repositories, with over 100 million open-source projects. This makes it a central hub for developers worldwide.
 - **Git hosting:** GitHub provides a space for developers to store and manage their code.
 - **Pull Requests (PRs), forks:** These features allow developers to propose changes and collaborate on projects.
 - **Issue tracking:** Developers can track bugs and feature requests, making project management more efficient.
 - **Code review:** GitHub facilitates peer review of code, ensuring quality and reliability.
 - **Collaboration:** It enables teams to work together seamlessly, regardless of location.
 - **Wiki:** Projects can have their own documentation, making it easier for others to understand and contribute.
 - **Actions (CI/CD):** GitHub Actions allows for continuous integration and deployment, automating workflows and improving efficiency.
- **"Forking a project"**
 - In open-source communities, "forking" can have a negative connotation as it might imply creating a competing project by modifying an existing one.
 - In GitHub terms, forking is a way to copy a project so you can make changes and contribute back, even if you don't have direct write access to the original project. This is a common practice to encourage collaboration and innovation.